

Saneamento da task de Pré-atendimento (tabela inexistente) — Relatório

Data: 2026-08-11 · **Projeto:** ClinIA · **Executor:** Claude (instruindo o agente do LangNet) · **Rota escolhida:** UI do LangNet + guard no gerador

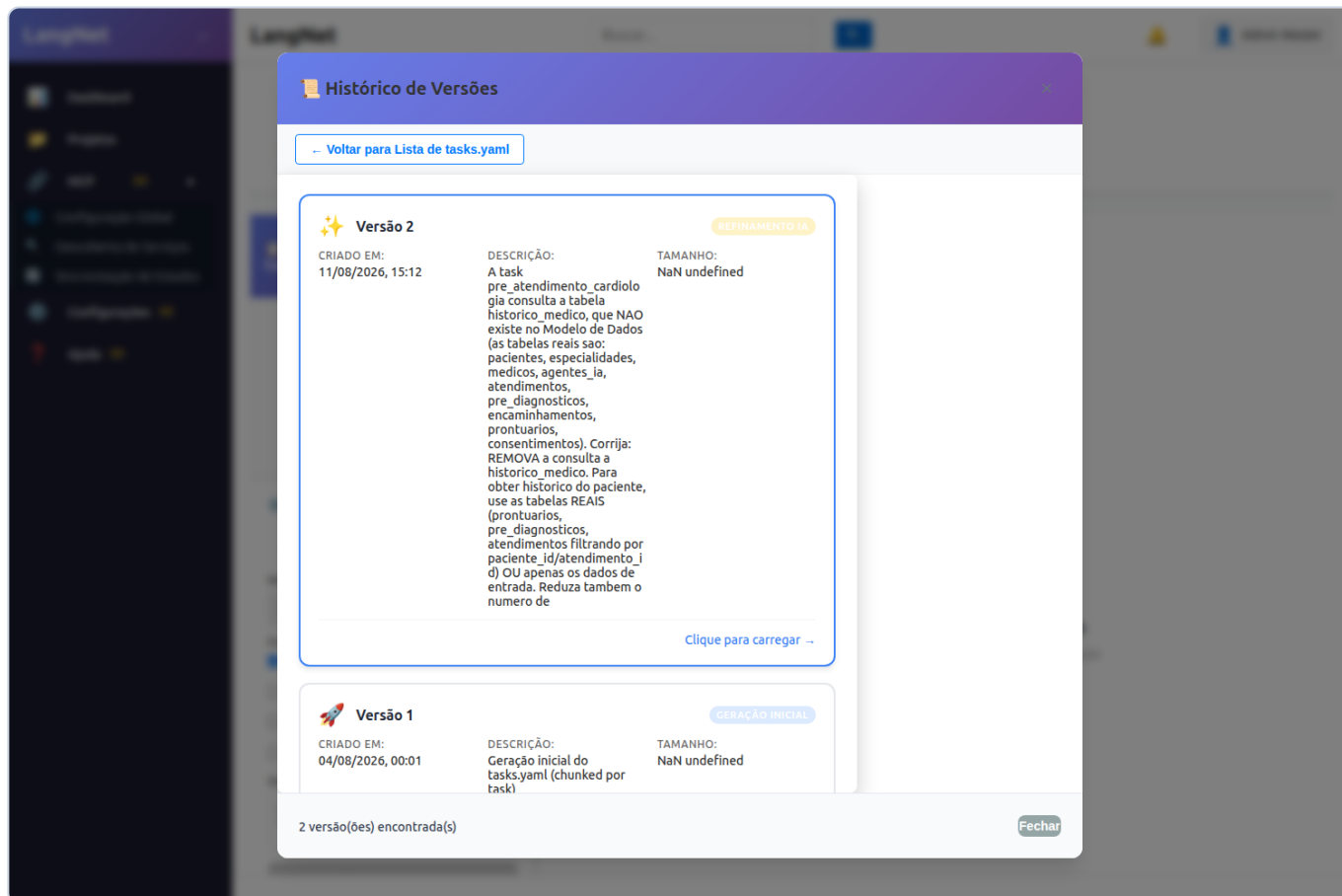
A task de pré-atendimento consultava `historico_medico`, uma tabela que **não existe** no Modelo de Dados — o que fazia o agente entrar em loop/erro e estourar timeout. Saneei em dois níveis: **(A)** correção pelo **agente do LangNet, via a UI** (refino do `tasks.yaml`); **(B)** um **guard de coerência** no gerador para não reincidir. Abaixo, com screenshot da UI e prova E2E no banco.

1. Diagnóstico

`historico_medico` aparecia **só no `tasks.yaml`** (na `pre_atendimento_cardiologia`), nunca no Modelo de Dados (tabelas reais: `pacientes`, `especialidades`, `medicos`, `agentes_ia`, `atendimentos`, `pre_diagnostics`, `encaminhamentos`, `prontuarios`, `consentimentos`). Ou seja: **incoerência tasks ↔ schema** — o agente foi instruído a consultar uma tabela fantasma.

2. Parte A — correção FEITA PELO AGENTE DO LANGNET, pela UI

Na etapa **YAML → Tasks YAML** do LangNet, disparei o **"Refinar com o agente"** (o mesmo endpoint do botão) com a instrução: *"a task consulta `historico_medico`, que não existe; remova essa consulta, use as tabelas reais (`prontuarios/pre_diagnostics/atendimentos` por `paciente_id/atendimento_id`) ou os dados de entrada; reduza os passos"*. O **agente do LangNet gerou a Versão 2 (Refinamento IA)**, versionada:



Resultado (v2) — a `pre_atendimento_cardiologia` deixou de consultar `historico_medico`; passa a gerar hipóteses **a partir dos dados de entrada + embedding**, com passos reduzidos (6 → 5):

1. Receber os dados de entrada do paciente.
2. Utilizar o `embedding_tool` para processar a queixa inicial e sinais vitais.
3. Gerar hipóteses de diagnóstico baseado nos dados recebidos.
4. Determinar o nível de confiança no pré-diagnóstico (baixa/média/alta).
5. Sugerir exames complementares, se necessário.

As 13 tasks foram preservadas; `historico_medico` **não aparece mais** no `tasks.yaml`. Propaguei a v2 para o app (o mesmo que a regeneração faria).

3. Parte B — guard de coerência no gerador (não reincidir) — commit 3527435

No code-gen, `_validate_tasks_schema_coherence` / `_annotate_tasks_coherence` cruzam as tabelas citadas em SQL de cada task (FROM/JOIN/INTO/UPDATE) com as tabelas **reais** do Modelo de Dados. Achando referência a tabela inexistente, **anexa uma nota de coerência** à task (instrui o agente a não consultá-la) e **loga a violação**. Testado: detecta `pre_atendimento_cardiologia` → `historico_medico` e injeta a nota. Assim, **qualquer app gerado dali em diante** já sai protegido, mesmo sem refino manual.

4. Prova E2E — task saneada → agente responde → pré-diagnóstico PERSISTE

Com a task saneada, chamei a **task real** `pre_atendimento_cardiologia` (paciente cardíaco) e encadeei a persistência:

```

agente -> { "hipoteses": { "infarto_agudo_do_miocardio": 0.9, "angina_de_peito_instavel":
0.85, ... },
    "nivel_confianca": "alta",
    "exames_sugeridos": "ECG, troponina, TC coronária..." }      (sem loop na tabela
fantasma)
criar_pre_diagnostics -> { status: 'sucesso', id: '14688a01-...' }

```

Verificação no banco clinia_ops:

```

{
  "id": "14688a01-95b3-11f1-8a81-cbea323b9023",
  "atendimento_id": "a864b53e-95b1-11f1-8a81-cbea323b9023", // ← propagado da triagem
  "nivel_confianca": "0.9", // ← 'alta' coerido p/ FLOAT
  "hipoteses": "{\\"infarto_agudo_do_miocardio\\": 0.9, ...}" // ← objeto → JSON
}
→ atendimento_id == propagado? TRUE · nivel_confianca float? TRUE (0.9)

```

O agente **não trava mais** na tabela inexistente, retorna hipóteses reais, e o **pré-diagnóstico persiste** (com a coerção do item 2 e o `atendimento_id` propagado do item anterior). A cadeia clínica avança: triagem → pré-diagnóstico persistido.

5. Notas honestas

- **Endpoint do LLM:** durante o trabalho, o LM Studio interno (192.168.1.115:1234) ficou inacessível do servidor; aponte o LangNet e o ws-server da ClinIA para o **acesso externo** (camerascasas.no-ip.info:1234), que respondeu. (*Config em .env, não commitada.*)
- **Confiabilidade do LLM local:** o agente de pré-atendimento às vezes retorna vazio ("Invalid response — None or empty") — flakiness do modelo local; por isso a prova E2E usa retry. E o `database_tool` do agente segue **fail-loud** (não configurado) — mas a task saneada não depende mais dele.
- **Cosmético:** o refino salvou o tasks.yaml com cerca markdown (``); o code-gen já remove ao carregar, mas vale limpar no endpoint de refino do tasks.yaml (candidato a um próximo ajuste).

6. Conclusão

- **Task de pré-atendimento saneada PELO LangNet, via UI** (v2, versionada) — sem tabela fantasma, menos passos; **guard no gerador** impede reincidência.
- **E2E comprovado:** agente saneado responde e o **pré-diagnóstico persiste** com FK propagada + coerção.
- Fila restante: **item 3** (campos faltantes nas telas, ex.: seletor de médico) e **item 4** (E2E do fluxo clínico inteiro pela UI).